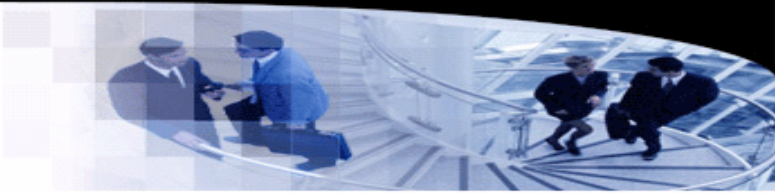
武漢大學
WUHAN UNIVERSITY

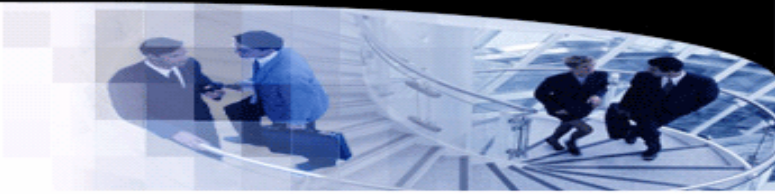


# Programming Languages and Compilers

## Lecture 27. Runtime Organization

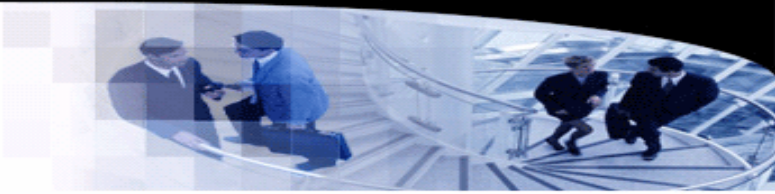
袁梦霆 (YUAN Mengting)
ymt@whu.edu.cn
School of Computer Science, Wuhan University

# Review of The Course

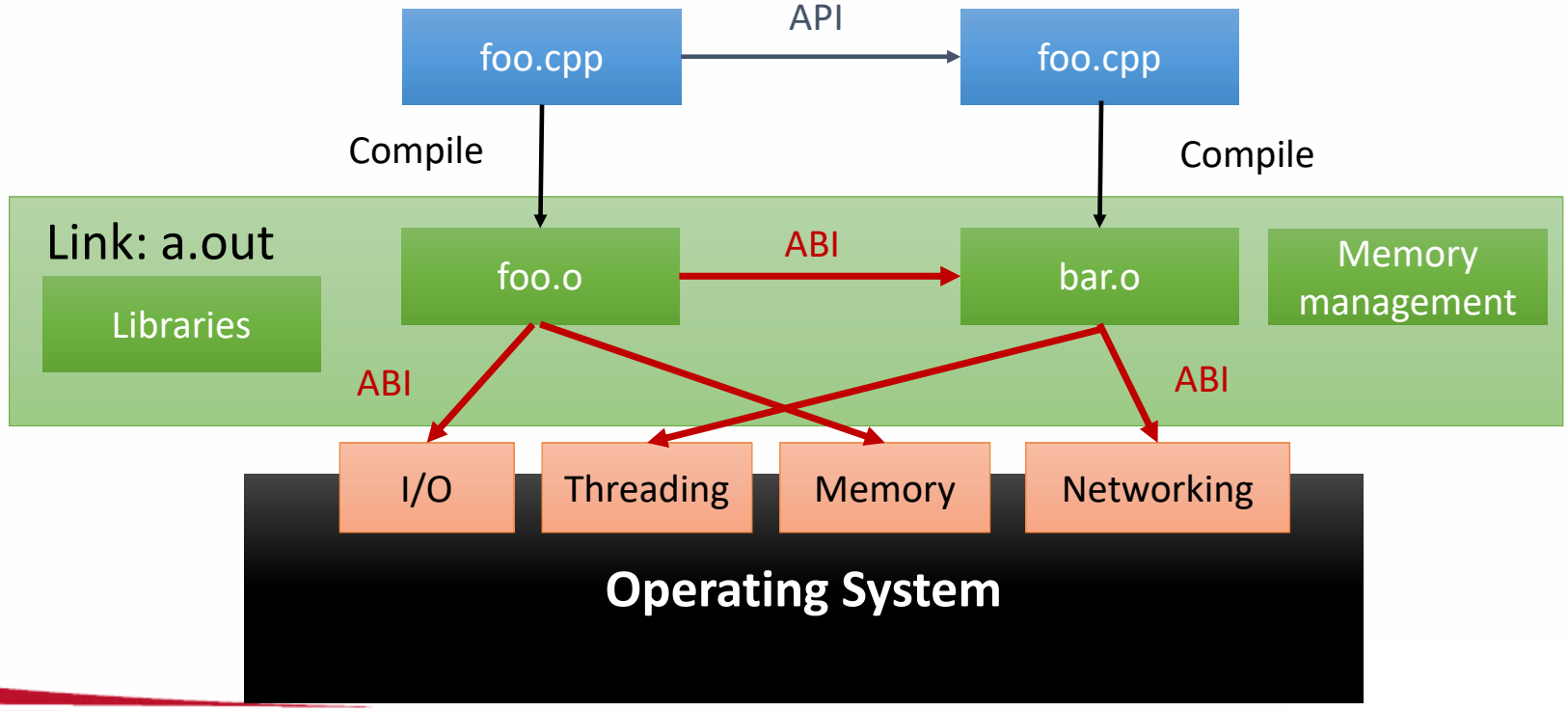


- What we have learned:
  - Given an expression-based language, we have learned to generate assembly programs by designing a scanner, a parser, a code generator.
  - The typical compilation flow is source code → AST → IR → assembly.
  - By adding control-flow statements to the language, the IR may be a CFG.
  - So far, the source program is written in a single module/procedure/function.
  - What if the program contains multiple modules?
  - How to translate the interactions (calls/returns) between these modules?
  - What if the program invokes a functionality (e.g., `printf`) of the operating system?

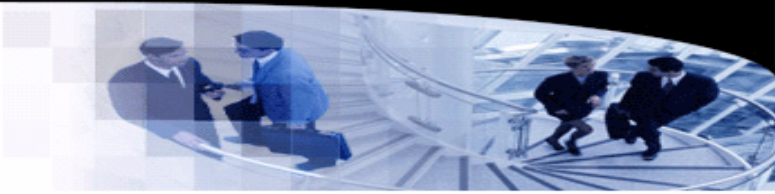
# Runtime Organization



- The goal of this lecture
  - Interactions between binary modules: ABI
  - Memory organization of binary programs.

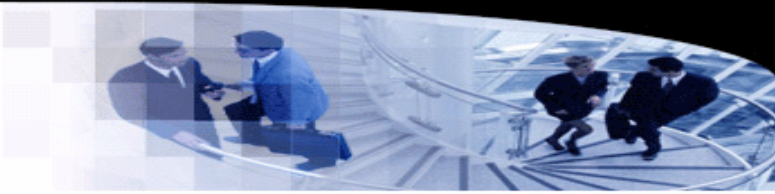


# Application Binary Interface

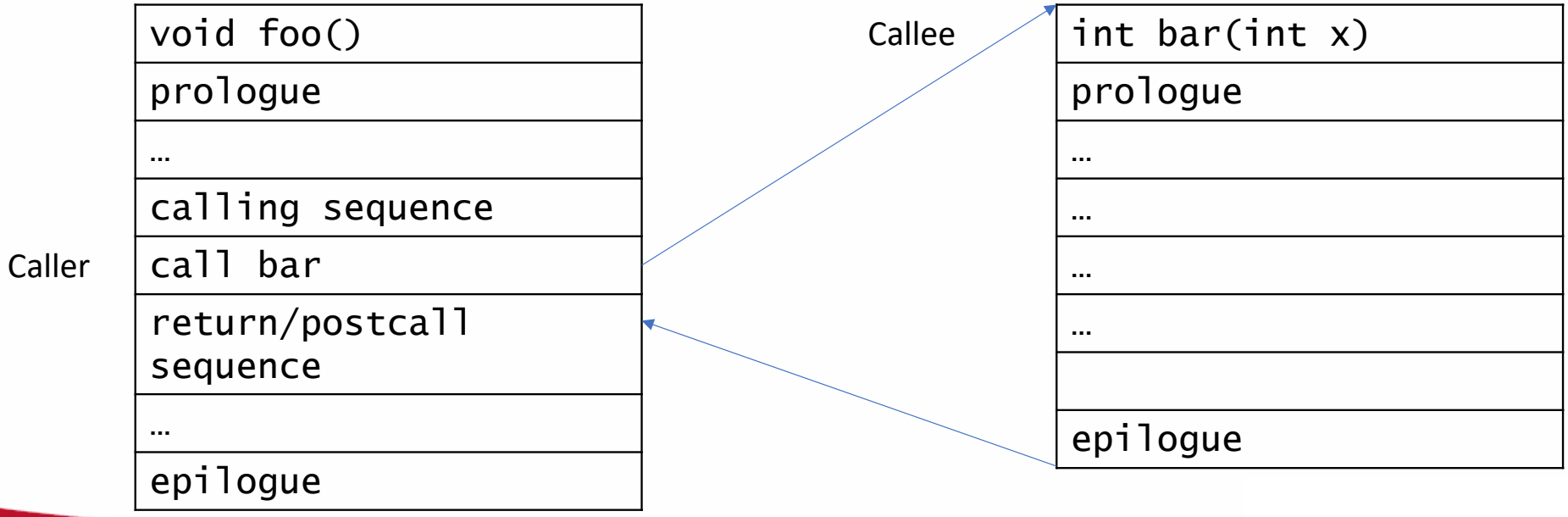


- Application binary interface (ABI)
  - An interface between two binary modules.
  - An ABI defines how to access data structures and modules at the level of machine codes.
  - It is often defined by compilers or operating systems.
- Aspects of ABI
  - Instruction set.
  - Data types, sizes, layouts, and alignments.
  - Calling convention.
  - System calls.
  - Binary formats.

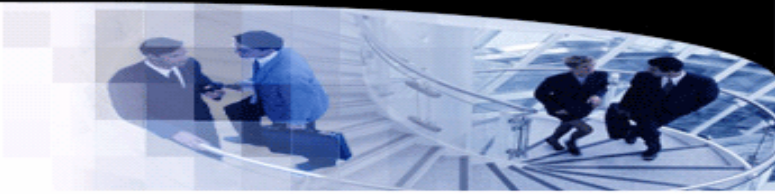
# Application Binary Interface



- Calling convention
  - A calling convention is a machine-code level protocol that defines how functions receives parameters from their callers and return values to their callers.
- Example of a function call

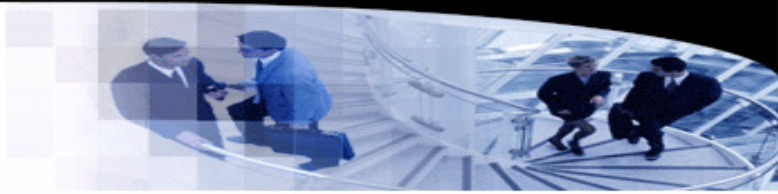


# Application Binary Interface



- Calling convention
  - A function **prologue** prepares the resources to run the function.
  - A typical prologue may
    - store the states of the caller to the stack, and
    - allocate a stack frame from the stack.
  - The **epilogue** of function reverses the action of the prologue, and returns control to the caller.
  - A typical epilogue may
    - free the current stack frame,
    - restore the states of the caller, and
    - return to the caller.

# Application Binary Interface

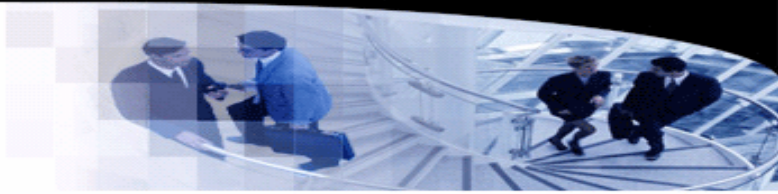


- Calling convention
  - Example of a function call.

```
int a, b;

int bar(int x, int y) {
  return x + y;
}

void foo() {
  a = bar(b, 3);
}
```

```
_Z3foov:                                # @_Z3foov
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)                      # 4-byte Folded Spill
        sw      s0, 8(sp)                       # 4-byte Folded Spill
        addi    s0, sp, 16
        lui     a0, %hi(b)
        lw      a0, %lo(b)(a0)
        li      a1, 3
        call    _Z3barii
        lui     a1, %hi(a)
        sw      a0, %lo(a)(a1)
        lw      ra, 12(sp)                      # 4-byte Folded Reload
        lw      s0, 8(sp)                       # 4-byte Folded Reload
        addi    sp, sp, 16
        ret
```

# Application Binary Interface



- Calling convention
  - Example of a function call.

```
int a, b;

int bar(int x, int y) {
  return x + y;
}

void foo() {
  a = bar(b, 3);
}
```

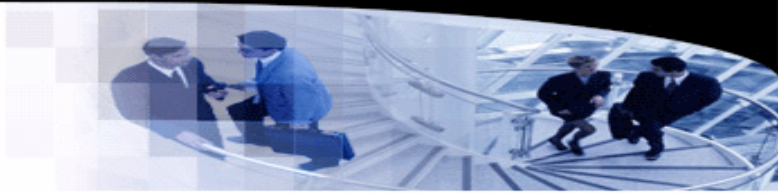
```
_Z3foov:                                # @_Z3foov
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)              # 4-byte Folded Spill
        sw      s0, 8(sp)               # 4-byte Folded Spill
        addi    s0, sp, 16
        lui     a0, %hi(b)
        lw      a0, %lo(b)(a0)
        li      a1, 3
        call    _Z3barii
        lui     a1, %hi(a)
        sw      a0, %lo(a)(a1)
        lw      ra, 12(sp)              # 4-byte Folded Reload
        lw      s0, 8(sp)               # 4-byte Folded Reload
        addi    sp, sp, 16
        ret
```

Prologue.

Epilogue.

Calling sequence.

# Application Binary Interface



- Calling convention
  - Example of a function call.

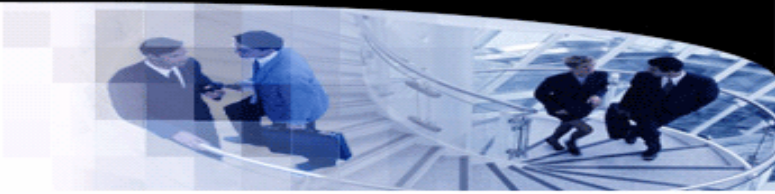
```c
int a, b;

int bar(int x, int y) {
  return x + y;
}

void foo() {
  a = bar(b, 3);
}
```

Allocate memory from the stack.

```
_Z3foov:                                # @_Z3foov
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)              # 4-byte Folded Spill
        sw      s0, 8(sp)               # 4-byte Folded Spill
        addi    s0, sp, 16
        lui     a0, %hi(b)
        lw      a0, %lo(b)(a0)
        li      a1, 3
        call    _Z3barii
        lui     a1, %hi(a)
        sw      a0, %lo(a)(a1)
        lw      ra, 12(sp)              # 4-byte Folded Reload
        lw      s0, 8(sp)               # 4-byte Folded Reload
        addi    sp, sp, 16
        ret
```

Free the memory.

# Application Binary Interface



- Calling convention
  - Example of a function call.

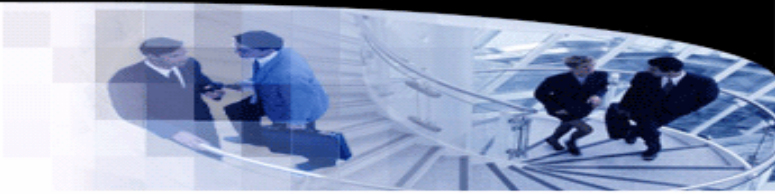
```c
int a, b;

int bar(int x, int y) {
  return x + y;
}

void foo() {
  a = bar(b, 3);
}
```

Store the registers.

```
_Z3foov:                                # @_Z3foov
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)              # 4-byte Folded Spill
        sw      s0, 8(sp)               # 4-byte Folded Spill
        addi    s0, sp, 16
        lui     a0, %hi(b)
        lw      a0, %lo(b)(a0)
        li      a1, 3
        call    _Z3barii
        lui     a1, %hi(a)
        sw      a0, %lo(a)(a1)
        lw      ra, 12(sp)              # 4-byte Folded Reload
        lw      s0, 8(sp)               # 4-byte Folded Reload
        addi    sp, sp, 16
        ret
```

Restore the registers.

# Application Binary Interface



- Calling convention
  - Example of a function call.

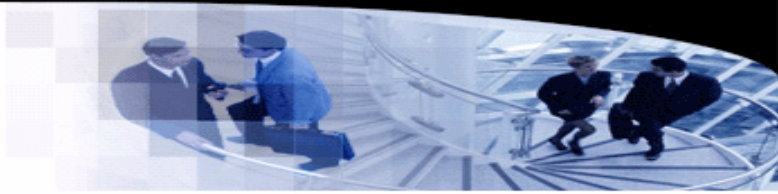
```c
int a, b;

int bar(int x, int y) {
  return x + y;
}

void foo() {
  a = bar(b, 3);
}
```

```
_Z3foov:                                # @_Z3foov
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)              # 4-byte Folded Spill
        sw      s0, 8(sp)               # 4-byte Folded Spill
        addi    s0, sp, 16
        lui     a0, %hi(b)
        lw      a0, %lo(b)(a0)
        li      a1, 3
        call    _Z3barii
        lui     a1, %hi(a)
        sw      a0, %lo(a)(a1)
        lw      ra, 12(sp)              # 4-byte Folded Reload
        lw      s0, 8(sp)               # 4-byte Folded Reload
        addi    sp, sp, 16
        ret
```

The first argument to pass.

The second argument.

# Application Binary Interface

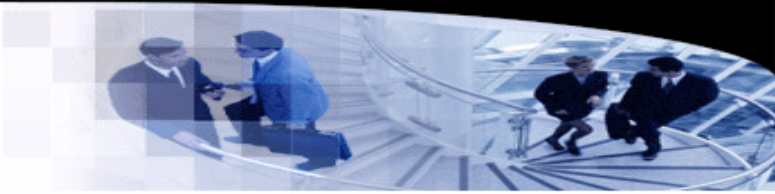


- Calling convention
  - Example of a function call.

```c
int a, b;

int bar(int x, int y) {
  return x + y;
}

void foo() {
  a = bar(b, 3);
}
```

```
_Z3barii:                               # @_Z3barii
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)                      # 4-byte Folded Spill
        sw      s0, 8(sp)                       # 4-byte Folded Spill
        addi    s0, sp, 16
        sw      a0, -12(s0)
        sw      a1, -16(s0)
        lw      a0, -12(s0)
        lw      a1, -16(s0)
        add     a0, a0, a1
        lw      ra, 12(sp)                      # 4-byte Folded Reload
        lw      s0, 8(sp)                       # 4-byte Folded Reload
        addi    sp, sp, 16
        ret
```

# Application Binary Interface



- Calling convention
  - Example of a function call.

```
int a, b;

int bar(int x, int y) {
  return x + y;
}

void foo() {
  a = bar(b, 3);
}
```

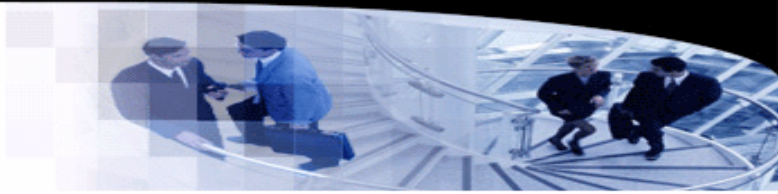
```
_Z3barii:                               # @_Z3barii
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)                      # 4-byte Folded Spill
        sw      s0, 8(sp)                       # 4-byte Folded Spill
        addi    s0, sp, 16
        sw      a0, -12(s0)
        sw      a1, -16(s0)
        lw      a0, -12(s0)
        lw      a1, -16(s0)
        add     a0, a0, a1
        lw      ra, 12(sp)                      # 4-byte Folded Reload
        lw      s0, 8(sp)                       # 4-byte Folded Reload
        addi    sp, sp, 16
        ret
```

The first parameter.

The second parameter.

The return value.

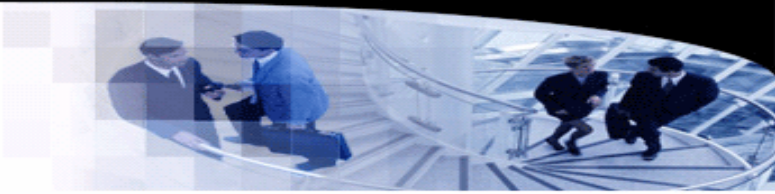
# Application Binary Interface



- RISCV ABI
  - https://riscv.org/wp-content/uploads/2024/12/riscv-calling.pdf
  - Data types

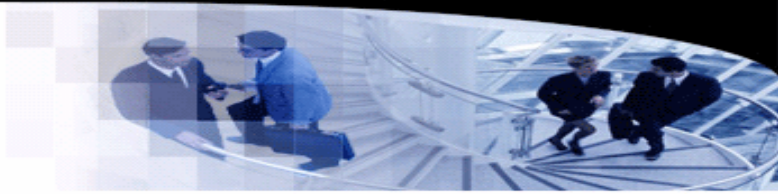
| C type | Description | Bytes in RV32 | Bytes in RV64 |
|---|---|---|---|
| `char` | Character value/byte | 1 | 1 |
| `short` | Short integer | 2 | 2 |
| `int` | Integer | 4 | 4 |
| `long` | Long integer | 4 | 8 |
| `long long` | Long long integer | 8 | 8 |
| `void*` | Pointer | 4 | 8 |
| `float` | Single-precision float | 4 | 4 |
| `double` | Double-precision float | 8 | 8 |
| `long double` | Extended-precision float | 16 | 16 |

# Application Binary Interface



- RISCV Calling convention
  - Pass arguments in registers when possible.
  - `a0` – `a7` for integer arguments, and `fa0` – `fa7` for floating-point arguments.
  - Values are returned from functions in integer registers `a0` and `a1` and floating-point registers `fa0` and `fa1`.
  - Larger return values are passed entirely in memory.
    - The caller allocates this memory and passes a pointer to it as an implicit first parameter to the callee.
  - The stack grows downward and the stack pointer is always kept 16-byte aligned.

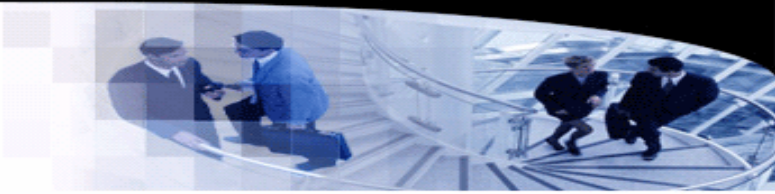
# Application Binary Interface



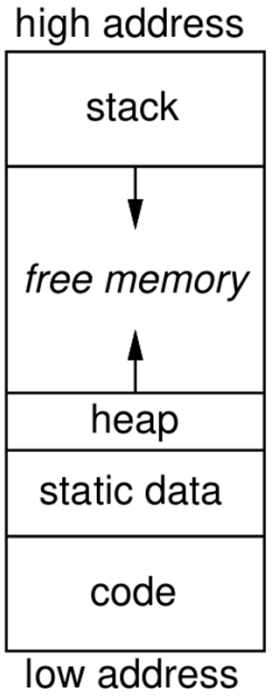
- RISCV Calling convention
  - t0 – t6 and ft0 – ft6 are temporary registers that are volatile across calls.
  - The caller saves these values if they are later used.

| Register | ABI Name | Description | Saver |
|---|---|---|---|
| x0 | zero | Hard-wired zero | — |
| x1 | ra | Return address | Caller |
| x2 | sp | Stack pointer | Callee |
| x3 | gp | Global pointer | — |
| x4 | tp | Thread pointer | — |
| x5–7 | t0–2 | Temporaries | Caller |
| x8 | s0/fp | Saved register/frame pointer | Callee |
| x9 | s1 | Saved register | Callee |
| x10–11 | a0–1 | Function arguments/return values | Caller |
| x12–17 | a2–7 | Function arguments | Caller |
| x18–27 | s2–11 | Saved registers | Callee |
| x28–31 | t3–6 | Temporaries | Caller |
| f0–7 | ft0–7 | FP temporaries | Caller |
| f8–9 | fs0–1 | FP saved registers | Callee |
| f10–11 | fa0–1 | FP arguments/return values | Caller |
| f12–17 | fa2–7 | FP arguments | Caller |
| f18–27 | fs2–11 | FP saved registers | Callee |
| f28–31 | ft8–11 | FP temporaries | Caller |

# Memory Organization

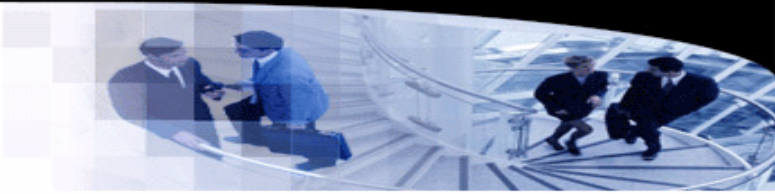


- Memory layout
  - The operating system is supposed to provide a logic block of memory.
  - It is the duty of the compiler to bind high-level names to low-level memory addresses in the block.
  - A typical memory layout:
  - However, the memory organizations of modern compilers are section based.




```
ymt@DESKTOP-VPDNSDJ:~/opt$ objdump -a -s a.out

a.out:     file format elf64-x86-64
a.out

Contents of section .interp:
 0318 2f6c6962 36342f6c 642d6c69 6e75782d  /lib64/ld-linux-
 0328 7838362d 36342e73 6f2e3200           x86-64.so.2.
Contents of section .note.gnu.property:
 0338 04000000 10000000 05000000 474e5500  ............GNU.
 0348 028000c0 04000000 01000000 00000000  ................
Contents of section .note.gnu.build-id:
 0358 04000000 14000000 03000000 474e5500  ............GNU.
 0368 a66f2460 e64eef00 4fce6726 2512267d  .o$`.N..O.g&%.&}
 0378 da2c1d59                             .,.Y
Contents of section .note.ABI-tag:
 037c 04000000 10000000 01000000 474e5500  ............GNU.
 038c 00000000 03000000 02000000 00000000  ................
Contents of section .gnu.hash:
 03a0 02000000 06000000 01000000 06000000  ................
 03b0 00008100 00000000 06000000 00000000  ................
 03c0 d165ce6d                             .e.m
Contents of section .dynsym:
 03c8 00000000 00000000 00000000 00000000  ................
 03d8 00000000 00000000 10000000 12000000  ................
```

# Memory Organization

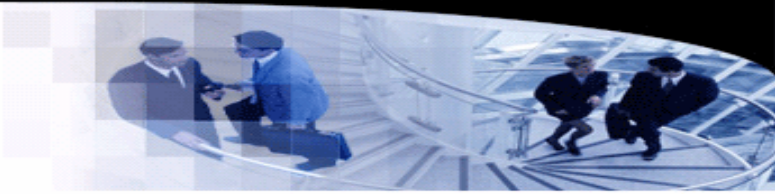


- Static allocation
  - Those variables whose address can be decided at compile-time.
  - Global variables.
  - Static variables.
  - Each of these variables is assigned to an address, which is an offset in a data section.
  - The real address of the data section is decided by the linker.

```
int a, b, c[10];

void f() {
  static int d = 0;
  a = 10;
}
```

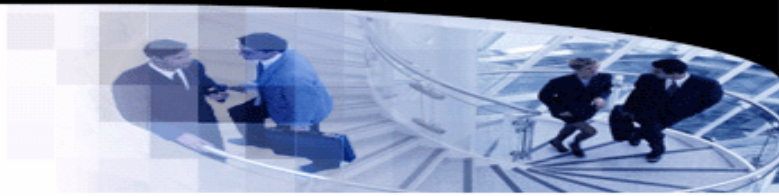
```
        .type   f.d,@object
        .section        .sbss,"aw",@nobits
        .p2align        2
f.d:
        .word   0
        .size   f.d, 4

        .type   a,@object
        .globl  a
        .p2align        2
a:
        .word   0
        .size   a, 4

        .type   b,@object
        .globl  b
        .p2align        2
b:
        .word   0
        .size   b, 4

        .type   c,@object
        .bss
        .globl  c
        .p2align        2
c:
        .zero   40
        .size   c, 40
```

# Memory Organization



- Stack frames
  - A function may have multiple running instances due to recursions.
  - Therefore, it is impossible to assign fixed addresses to local variables.
  - In this case, each instance allocates a new **stack frame** to store its own information.
  - Although the address of stack frame is not fixed, the offset of local variable in the stack frame is fixed.
  - Hence, local variables are assigned to addresses in stack frames.

# Memory Organization

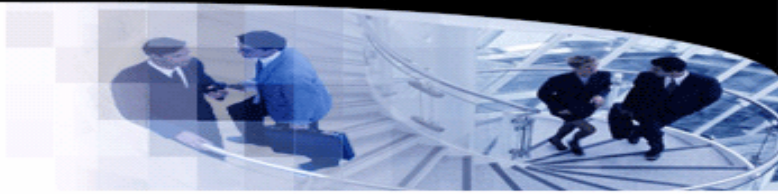


- Stack frames
  - A stack frame may store parameters, states, local variables, return address, and temporary variables.

```
int a, b;

int bar(int x, int y) {
  return x + y;
}

void foo() {
  a = bar(b, 3);
}
```

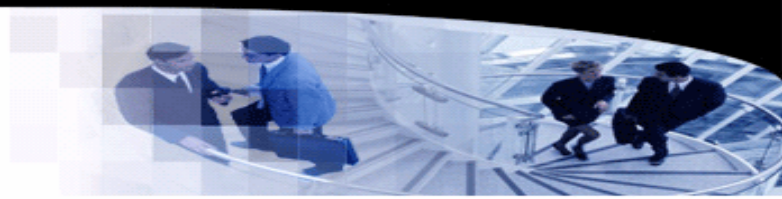
```
_Z3barii:                               # @_Z3barii
# %bb.0:
        addi    sp, sp, -16
        sw      ra, 12(sp)                      # 4-byte Folded Spill
        sw      s0, 8(sp)                       # 4-byte Folded Spill
        addi    s0, sp, 16
        sw      a0, -12(s0)
        sw      a1, -16(s0)
        lw      a0, -12(s0)
        lw      a1, -16(s0)
        add     a0, a0, a1
        lw      ra, 12(sp)                      # 4-byte Folded Reload
        lw      s0, 8(sp)                       # 4-byte Folded Reload
        addi    sp, sp, 16
        ret
```

# Memory Organization



- Stack frames
  - Given an OCaml function with its environment (closure), how can this function access the variables in the environment?
    - `let x = 1 in`
    - `  let y = 2 in`
    - `    let add m n = m + n in`
    - `      add x y;;`

# Memory Organization



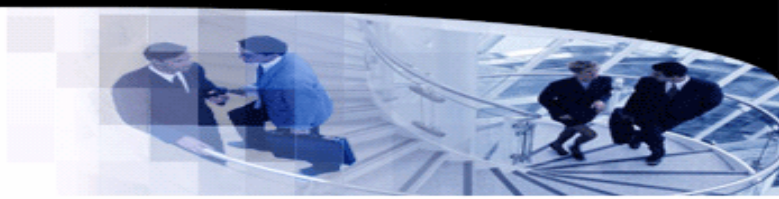
- Stack frames
  - The caller prepares a closure for the callee.

```
int a;

extern void f();

void f(int b) {
  int c = 10;
  a = [=](int n) { return n + b + c; } (a);
}
```

```
        .type   _Z1fi,@function
_Z1fi:
        .cfi_startproc
        addi    sp, sp, -32
        .cfi_def_cfa_offset 32
        sw      ra, 28(sp)
        sw      s0, 24(sp)
        .cfi_offset ra, -4
        .cfi_offset s0, -8
        addi    s0, sp, 32
        .cfi_def_cfa s0, 0
        sw      a0, -12(s0)
        li      a0, 10
        sw      a0, -16(s0)
        lw      a0, -12(s0)
        sw      a0, -24(s0)
        lw      a0, -16(s0)
        sw      a0, -20(s0)
        lui     a0, %hi(a)
        sw      a0, -28(s0)
        lw      a1, %lo(a)(a0)
        addi    a0, s0, -24
        call    _ZZ1fiENK3$_0clEi
        lw      a1, -28(s0)
        sw      a0, %lo(a)(a1)
        lw      ra, 28(sp)
        lw      s0, 24(sp)
        addi    sp, sp, 32
        ret
```

```
        .type   _ZZ1fiENK3$_0clEi,@function
_ZZ1fiENK3$_0clEi:
        addi    sp, sp, -16
        sw      ra, 12(sp)
        sw      s0, 8(sp)
        addi    s0, sp, 16
        sw      a0, -12(s0)
        sw      a1, -16(s0)
        lw      a1, -12(s0)
        lw      a0, -16(s0)
        lw      a2, 0(a1)
        add     a0, a0, a2
        lw      a1, 4(a1)
        add     a0, a0, a1
        lw      ra, 12(sp)
        lw      s0, 8(sp)
        addi    sp, sp, 16
        ret
```

# Memory Organization



- Heap
  - Heap is a piece of memory that offers dynamic allocations and deallocations.
  - User-managed heap:
    - Provide functions for user to allocate or free objects (e.g., `malloc` and `free`).
    - Disadvantages: Error prone.
  - Automatic-managed heap:
    - Garbage collection.
    - Less error pone.
    - Disadvantages: Performance issue.